

STUDENT NAME: **YASH CHAUHAN**

STUDENT ROLL NUMBER: **28**

SECTION: **CSE C-1**

STUDENT REG NUMBER: **230905194**

WEEK NO: **9**

DATE: **25 TH March 2026**

Lab No. 9 – DataBases – Part1

Q1. Design a web site using Django, which is a website directory – A site containing

links to other websites. A web page has different categories.

- A category table has a name, number of visits, and number of likes.
- A page table refers to a category, has a title, URL, and many views.

Design a form that populates the above database and displays it.

Model.py

```
from django.db import models

class Category(models.Model):
    """Model for website categories"""
    name = models.CharField(max_length=200, unique=True)
    visits = models.IntegerField(default=0)
    likes = models.IntegerField(default=0)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        verbose_name_plural = "Categories"
        ordering = ['-likes', '-visits']

    def __str__(self):
        return self.name

class Page(models.Model):
```

```

"""Model for website pages/links"""
category = models.ForeignKey(Category, on_delete=models.CASCADE,
related_name='pages')
title = models.CharField(max_length=200)
url = models.URLField()
views = models.IntegerField(default=0)
created_at = models.DateTimeField(auto_now_add=True)

class Meta:
ordering = ['-views', '-created_at']

def __str__(self):
return f"{self.title} ({self.category.name})"

```

Views.py

```

from django.shortcuts import render, redirect, get_object_or_404
from django.views.generic import ListView, DetailView, CreateView, UpdateView,
DeleteView
from django.urls import reverse_lazy
from django.contrib import messages
from django.db import models
from .models import Category, Page
from .forms import CategoryForm, PageForm

# Category Views
class CategoryListView(ListView):
    """Display all categories"""
    model = Category
    template_name = 'directory/category_list.html'
    context_object_name = 'categories'
    paginate_by = 10

class CategoryDetailView(DetailView):
    """Display pages in a specific category"""
    model = Category
    template_name = 'directory/category_detail.html'
    context_object_name = 'category'

def get_context_data(self, **kwargs):

```

```

context = super().get_context_data(**kwargs)
context['pages'] = self.object.pages.all()
return context

class CategoryCreateView(CreateView):
    """Create a new category"""
    model = Category
    form_class = CategoryForm
    template_name = 'directory/category_form.html'
    success_url = reverse_lazy('category_list')

    def form_valid(self, form):
        messages.success(self.request, f'Category "{form.cleaned_data["name"]}" created successfully!')
        return super().form_valid(form)

class CategoryUpdateView(UpdateView):
    """Update a category"""
    model = Category
    form_class = CategoryForm
    template_name = 'directory/category_form.html'
    success_url = reverse_lazy('category_list')

    def form_valid(self, form):
        messages.success(self.request, f'Category "{form.cleaned_data["name"]}" updated successfully!')
        return super().form_valid(form)

class CategoryDeleteView(DeleteView):
    """Delete a category"""
    model = Category
    template_name = 'directory/category_confirm_delete.html'
    success_url = reverse_lazy('category_list')

    def delete(self, request, *args, **kwargs):
        messages.success(request, f'Category deleted successfully!')
        return super().delete(request, *args, **kwargs)

```

```

# Page Views
class PageListView(ListView):
    """Display all pages"""
    model = Page
    template_name = 'directory/page_list.html'
    context_object_name = 'pages'
    paginate_by = 15

class PageDetailView(DetailView):
    """Display page details"""
    model = Page
    template_name = 'directory/page_detail.html'
    context_object_name = 'page'

# Dashboard view
def index(request):
    """Dashboard view showing statistics"""
    categories_count = Category.objects.count()
    pages_count = Page.objects.count()
    total_visits = Category.objects.values_list('visits',
        flat=True).aggregate(sum=models.Sum('visits'))['sum'] or 0
    total_likes = Category.objects.aggregate(sum=models.Sum('likes'))['sum'] or 0
    context = {
        'categories_count': categories_count,
        'pages_count': pages_count,
        'total_visits': total_visits,
        'total_likes': total_likes,
        'categories': Category.objects.all()[:5],
        'pages': Page.objects.all()[:5],
    }
    return render(request, 'directory/index.html', context)

```

Urls.py

```

"""
URL configuration for directory_project project.
"""
from django.contrib import admin
from django.urls import path, include

urlpatterns = [

```

```
path('admin/', admin.site.urls),
path('', include('directory.urls')),
]
```

Settings.py

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'directory',
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'directory_project.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [BASE_DIR / 'directory' / 'templates'],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]
```

```
WSGI_APPLICATION = 'directory_project.wsgi.application'
```

```
# Database
```

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.sqlite3',  
        'NAME': BASE_DIR / 'db.sqlite3',  
    }  
}
```

Index.html

```
{% extends 'directory/base.html' %}  
  
{% block title %}Dashboard - Web Directory{% endblock %}  
  
{% block content %}  
    <div class="dashboard">  
        <h1>Web Directory Dashboard</h1>  
        <p class="subtitle">Manage your website categories and pages</p>  
  
        <div class="stats-grid">  
            <div class="stat-card">  
                <div class="stat-value">{{ categories_count }}</div>  
                <div class="stat-label">Total Categories</div>  
            </div>  
            <div class="stat-card">  
                <div class="stat-value">{{ pages_count }}</div>  
                <div class="stat-label">Total Pages</div>  
            </div>  
            <div class="stat-card">  
                <div class="stat-value">{{ total_visits }}</div>  
                <div class="stat-label">Total Visits</div>  
            </div>  
            <div class="stat-card">  
                <div class="stat-value">{{ total_likes }}</div>  
                <div class="stat-label">Total Likes</div>  
            </div>  
        </div>  
  
        <div class="dashboard-sections">  
            <div class="section">
```

```

<div class="section-header">
<h2>Recent Categories</h2>
<a href="{% url 'category_list' %}" class="link-btn">View All →</a>
</div>
{% if categories %}
<div class="list-items">
{% for category in categories %}
<div class="list-item">
<div class="item-info">
<h3><a href="{% url 'category_detail'
category.pk %}">{{ category.name }}</a></h3>
<p class="item-meta">{{ category.pages.count }} pages | {{ category.visits }}
visits | {{ category.likes }} likes</p>
</div>
<div class="item-actions">
<a href="{% url 'category_update' category.pk %}" class="btn-small">Edit</a>
<a href="{% url 'category_delete' category.pk %}" class="btn-small btn-
danger">Delete</a>
</div>
</div>
{% endfor %}
</div>
{% else %}
<p class="empty-message">No categories yet. <a href="{% url
'category_create' %}">Create one</a></p>
{% endif %}
</div>

<div class="section">
<div class="section-header">
<h2>Recent Pages</h2>
<a href="{% url 'page_list' %}" class="link-btn">View All →</a>
</div>
{% if pages %}
<div class="list-items">
{% for page in pages %}
<div class="list-item">
<div class="item-info">
<h3><a href="{% url 'page_detail' page.pk %}">{{ page.title }}</a></h3>
<p class="item-meta">{{ page.category.name }} | {{ page.views }} views</p>
</div>
<div class="item-actions">
<a href="{% url 'page_update' page.pk %}" class="btn-small">Edit</a>
<a href="{% url 'page_delete' page.pk %}" class="btn-small btn-danger">Delete</a>

```

```

</div>
</div>
{% endfor %}
</div>
{% else %}
<p class="empty-message">No pages yet. <a href="{% url 'page_create' %}">Create
one</a></p>
{% endif %}
</div>
</div>
</div>
{% endblock %}

```

Base.html

```

{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>{% block title %}Web Directory{% endblock %}</title>
<link rel="stylesheet" href="{% static 'css/style.css' %}">
</head>
<body>
<header>
<nav class="navbar">
<div class="nav-container">
<div class="nav-logo">
<a href="{% url 'index' %}">📖 Web Directory</a>
</div>
<ul class="nav-menu">
<li><a href="{% url 'index' %}" class="nav-link">Dashboard</a></li>
<li><a href="{% url 'category_list' %}" class="nav-link">Categories</a></li>
<li><a href="{% url 'page_list' %}" class="nav-link">Pages</a></li>
<li><a href="{% url 'category_create' %}" class="nav-link nav-add">+ Add
Category</a></li>
<li><a href="{% url 'page_create' %}" class="nav-link nav-add">+ Add
Page</a></li>
</ul>
</div>
</nav>
</header>

```



```

<main>
<div class="container">
{% if messages %}
<div class="messages">
{% for message in messages %}
<div class="alert alert-{{ message.tags }}">{{ message }}</div>
{% endfor %}
</div>
{% endif %}

{% block content %}{% endblock %}
</div>
</main>

<footer>
<p>&copy; 2026 Web Directory. All rights reserved.</p>
</footer>
</body>
</html>

```

Forms.py

```

from django import forms
from .models import Category, Page

class CategoryForm(forms.ModelForm):
    """Form for creating and updating categories"""
    class Meta:
        model = Category
        fields = ['name', 'visits', 'likes']
        widgets = {
            'name': forms.TextInput(attrs={
                'class': 'form-control',
                'placeholder': 'Enter category name',
                'required': True
            }),
            'visits': forms.NumberInput(attrs={
                'class': 'form-control',
                'placeholder': 'Number of visits',
                'min': '0'
            }),

```

```
'likes': forms.NumberInput(attrs={
    'class': 'form-control',
    'placeholder': 'Number of likes',
    'min': '0'
}),
}

class PageForm(forms.ModelForm):
    """Form for creating and updating pages"""
    class Meta:
        model = Page
        fields = ['category', 'title', 'url', 'views']
        widgets = {
            'category': forms.Select(attrs={
                'class': 'form-control',
            }),
            'title': forms.TextInput(attrs={
                'class': 'form-control',
                'placeholder': 'Enter page title',
                'required': True
            }),
            'url': forms.URLInput(attrs={
                'class': 'form-control',
                'placeholder': 'https://example.com',
                'required': True
            }),
            'views': forms.NumberInput(attrs={
                'class': 'form-control',
                'placeholder': 'Number of views',
                'min': '0'
            }),
        }
}
```

Django administration interface showing the 'Select page to change' view. The interface includes a sidebar with navigation links for Groups, Users, Categories, and Pages. The main content area displays a table of pages with columns for Title, Category, Views, and Created At. The table lists 17 pages, including LPG Gas, Wikipedia, CNN, BBC News, Khan Academy, Udemy, Coursera, Visual Studio Code, Stack Overflow, GitHub, Instagram, LinkedIn, Twitter, Facebook, DuckDuckGo, Bing, and Google. A filter sidebar on the right allows filtering by category and by created at date.

TITLE	CATEGORY	VIEWS	CREATED AT
LPG Gas	LPG Gas	18	March 25, 2026, 9:37 a.m.
Wikipedia	News & Information	920	March 25, 2026, 9:31 a.m.
CNN	News & Information	180	March 25, 2026, 9:31 a.m.
BBC News	News & Information	200	March 25, 2026, 9:31 a.m.
Khan Academy	Learning Platforms	280	March 25, 2026, 9:31 a.m.
Udemy	Learning Platforms	450	March 25, 2026, 9:31 a.m.
Coursera	Learning Platforms	350	March 25, 2026, 9:31 a.m.
Visual Studio Code	Development Tools	420	March 25, 2026, 9:31 a.m.
Stack Overflow	Development Tools	880	March 25, 2026, 9:31 a.m.
GitHub	Development Tools	750	March 25, 2026, 9:31 a.m.
Instagram	Social Media	900	March 25, 2026, 9:31 a.m.
LinkedIn	Social Media	600	March 25, 2026, 9:31 a.m.
Twitter	Social Media	800	March 25, 2026, 9:31 a.m.
Facebook	Social Media	1222	March 25, 2026, 9:31 a.m.
DuckDuckGo	Search Engines	80	March 25, 2026, 9:31 a.m.
Bing	Search Engines	150	March 25, 2026, 9:31 a.m.
Google	Search Engines	501	March 25, 2026, 9:31 a.m.

Web Directory Dashboard interface showing a summary of website categories and pages. The dashboard includes a header with navigation links for Dashboard, Categories, Pages, Add Category, and Add Page. The main content area displays four summary cards: 6 Total Categories, 17 Total Pages, 705 Total Visits, and 268 Total Likes. Below the summary cards, there are sections for Recent Categories and Recent Pages, each listing the category/page name, page count, visits, and likes, along with links to view, edit, and delete.

Web Directory Dashboard

Manage your website categories and pages

6
Total Categories

17
Total Pages

705
Total Visits

268
Total Likes

Recent Categories

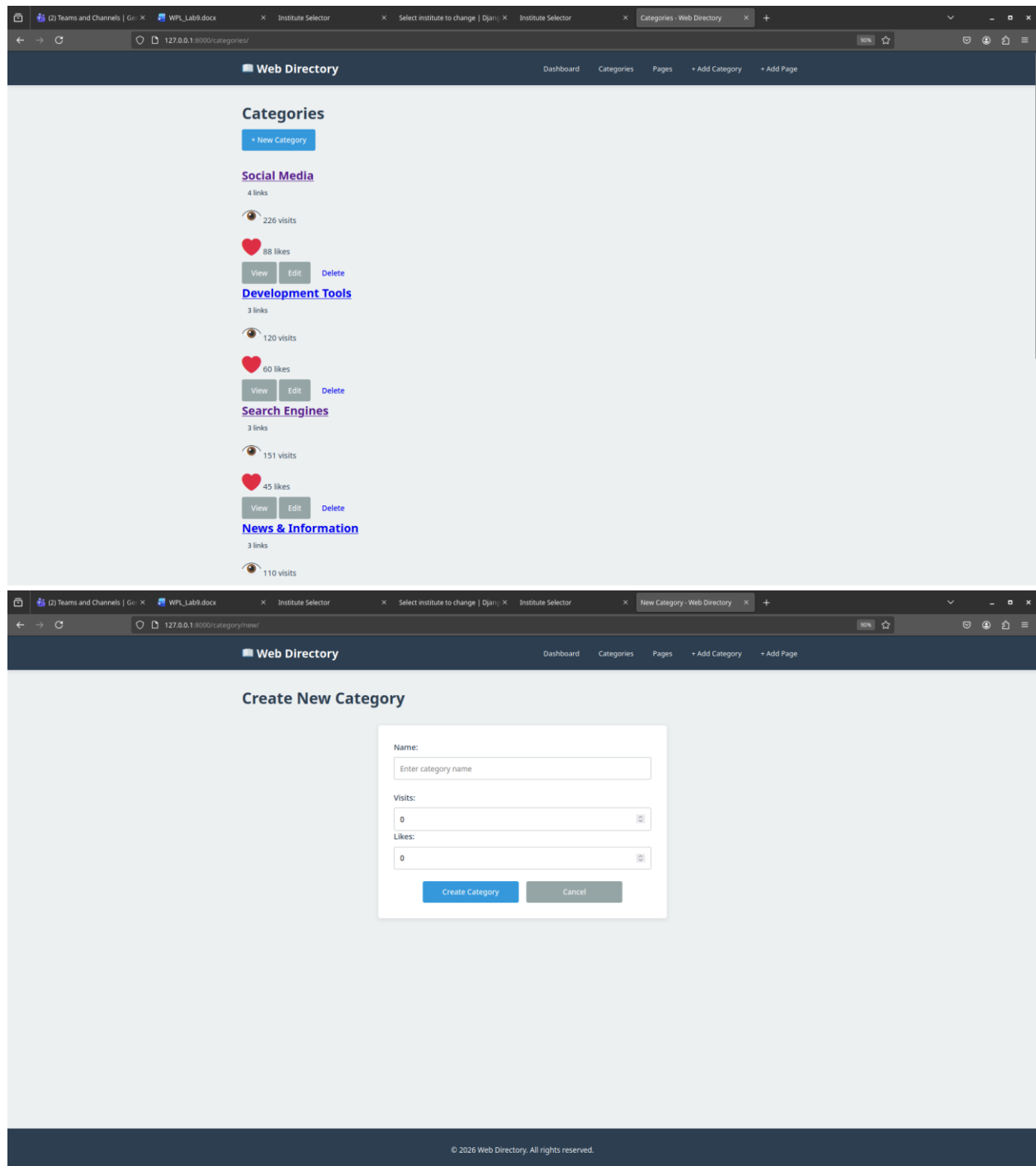
[View All](#)

- Social Media**
4 pages | 226 visits | 88 likes
[Edit](#) [Delete](#)
- Development Tools**
3 pages | 120 visits | 60 likes
[Edit](#) [Delete](#)
- Search Engines**
3 pages | 151 visits | 45 likes
[Edit](#) [Delete](#)
- News & Information**
3 pages | 110 visits | 40 likes
[Edit](#) [Delete](#)
- Learning Platforms**
3 pages | 90 visits | 35 likes
[Edit](#) [Delete](#)

Recent Pages

[View All](#)

- Facebook**
Social Media | 1222 views
[Edit](#) [Delete](#)
- Wikipedia**
News & Information | 920 views
[Edit](#) [Delete](#)



Q2. Consider the following tables:

WORKS(person-name,Company-name,Salary)

LIVES(Person_name, Street, City)

Assume Table data suitably. Design a Django webpage and include an option to

insert data into WORKS table by accepting data from the user using TextBoxes.

Also, include an option to retrieve the names of people who work for a particular company along with the cities they live in (particular company name must be accepted from the user).

Urls.py

```
"""
URL configuration for employee_project project.
"""
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('employees.urls')),
]
```

Urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.index, name='index'),
    path('insert-works/', views.insert_works, name='insert_works'),
    path('insert-lives/', views.insert_lives, name='insert_lives'),
    path('search-company/', views.search_company, name='search_company'),
    path('view-works/', views.view_all_works, name='view_all_works'),
    path('view-lives/', views.view_all_lives, name='view_all_lives'),
]
```

Model.py

```
from django.db import models

class Works(models.Model):
    """WORKS table: Person employment information"""
    person_name = models.CharField(max_length=100)
    company_name = models.CharField(max_length=100)
```

```

salary = models.DecimalField(max_digits=10, decimal_places=2)
created_at = models.DateTimeField(auto_now_add=True)

class Meta:
    verbose_name_plural = "Works"
    ordering = ['company_name', 'person_name']

def __str__(self):
    return f"{self.person_name} - {self.company_name} (${self.salary})"

class Lives(models.Model):
    """LIVES table: Person location information"""
    works = models.OneToOneField(Works, on_delete=models.CASCADE,
    related_name='location')
    street = models.CharField(max_length=100)
    city = models.CharField(max_length=50)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        verbose_name_plural = "Lives"
        ordering = ['works__person_name']

    def __str__(self):
        return f"{self.works.person_name} - {self.city}"

    @property
    def person_name(self):
        """Get person name from related Works record"""
        return self.works.person_name

```

Forms.py

```

from django import forms
from .models import Works, Lives

class WorksForm(forms.ModelForm):
    """Form for inserting data into WORKS table"""

    class Meta:
        model = Works
        fields = ['person_name', 'company_name', 'salary']
        widgets = {

```

```

'person_name': forms.TextInput(attrs={
'class': 'form-control',
'placeholder': 'Enter person name',
'required': True
}),
'company_name': forms.TextInput(attrs={
'class': 'form-control',
'placeholder': 'Enter company name',
'required': True
}),
'salary': forms.NumberInput(attrs={
'class': 'form-control',
'placeholder': 'Enter salary (e.g., 50000.00)',
'step': '0.01',
'min': '0'
}),
}

```

```

class LivesForm(forms.ModelForm):
    """Form for inserting data into LIVES table"""
    class Meta:
        model = Lives
        fields = ['works', 'street', 'city']
        widgets = {
            'works': forms.Select(attrs={
                'class': 'form-control',
                'required': True
            }),
            'street': forms.TextInput(attrs={
                'class': 'form-control',
                'placeholder': 'Enter street address',
                'required': True
            }),
            'city': forms.TextInput(attrs={
                'class': 'form-control',
                'placeholder': 'Enter city name',
                'required': True
            }),
        }
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        # Show employee name and company in the dropdown

```

```

self.fields['works'].queryset =
Works.objects.select_related().order_by('person_name')
self.fields['works'].label_from_instance = lambda obj: f"{obj.person_name} at
{obj.company_name} (${obj.salary})"

class CompanySearchForm(forms.Form):
    """Form for searching employees by company"""
    company_name = forms.CharField(
        max_length=100,
        widget=forms.TextInput(attrs={
            'class': 'form-control',
            'placeholder': 'Enter company name to search',
            'required': True
        })
    )

```

Views.py

```

from django.shortcuts import render, redirect
from django.contrib import messages
from django.db.models import Q
from .models import Works, Lives
from .forms import WorksForm, LivesForm, CompanySearchForm

def index(request):
    """Dashboard view"""
    works_count = Works.objects.count()
    lives_count = Lives.objects.count()
    companies = Works.objects.values_list('company_name',
        flat=True).distinct().count()
    context = {
        'works_count': works_count,
        'lives_count': lives_count,
        'companies': companies,
    }
    return render(request, 'employees/index.html', context)

def insert_works(request):
    """Insert data into WORKS table"""
    if request.method == 'POST':

```



```

form = WorksForm(request.POST)
if form.is_valid():
    form.save()
    messages.success(
        request,
        f'Employee "{form.cleaned_data["person_name"]}" added to
        {form.cleaned_data["company_name"]}!'
    )
    return redirect('insert_works')
else:
    form = WorksForm()
    works_list = Works.objects.all().order_by('-created_at')[:10]
    context = {
        'form': form,
        'recent_works': works_list,
    }
    return render(request, 'employees/insert_works.html', context)

def insert_lives(request):
    """Insert data into LIVES table"""
    if request.method == 'POST':
        form = LivesForm(request.POST)
        if form.is_valid():
            form.save()
            works = form.cleaned_data['works']
            messages.success(
                request,
                f'Location for "{works.person_name}" ({works.company_name}) added!'
            )
            return redirect('insert_lives')
        else:
            form = LivesForm()
            lives_list = Lives.objects.select_related('works').order_by('-created_at')[:10]
            context = {
                'form': form,
                'recent_lives': lives_list,
            }
            return render(request, 'employees/insert_lives.html', context)

def search_company(request):
    """Search employees by company and display their cities"""

```

```

results = None
search_query = None
form = CompanySearchForm()
if request.method == 'POST':
    form = CompanySearchForm(request.POST)
if form.is_valid():
    search_query = form.cleaned_data['company_name']
    # Get all employees from the company
    works_records = Works.objects.filter(
        company_name__icontains=search_query
    ).order_by('person_name')
    # For each employee, check if they have a location record
    results = []
    for work in works_records:
        # Use the one-to-one relationship
        try:
            location = work.location
            results.append({
                'person_name': work.person_name,
                'company_name': work.company_name,
                'salary': work.salary,
                'city': location.city,
                'street': location.street,
            })
        except Lives.DoesNotExist:
            # If no location info, still include the employee
            results.append({
                'person_name': work.person_name,
                'company_name': work.company_name,
                'salary': work.salary,
                'city': 'Not available',
                'street': 'Not available',
            })
    if not results:
        messages.warning(request, f'No employees found for "{search_query}"')
    context = {
        'form': form,
        'results': results,
        'search_query': search_query,
    }
    return render(request, 'employees/search_company.html', context)

def view_all_works(request):

```

```

"""View all WORKS records"""
works_list = Works.objects.all().order_by('-created_at')
context = {
    'works_list': works_list,
}
return render(request, 'employees/view_all_works.html', context)

def view_all_lives(request):
    """View all LIVES records"""
    lives_list = Lives.objects.select_related('works').order_by('-created_at')
    context = {
        'lives_list': lives_list,
    }
    return render(request, 'employees/view_all_lives.html', context)

```

Settings.py

```

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'employees',
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'employee_project.urls'

TEMPLATES = [
    {

```

```

'BACKEND': 'django.template.backends.django.DjangoTemplates',
'DIRS': [BASE_DIR / 'employees' / 'templates'],
'APP_DIRS': True,
'OPTIONS': {
    'context_processors': [
        'django.template.context_processors.debug',
        'django.template.context_processors.request',
        'django.contrib.auth.context_processors.auth',
        'django.contrib.messages.context_processors.messages',
    ],
},
},
]

```

```
WSGI_APPLICATION = 'employee_project.wsgi.application'
```

```

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}

```

Base.html

```

{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>{% block title %}Employee Management{% endblock %}</title>
<link rel="stylesheet" href="{% static 'css/style.css' %}">
</head>
<body>
<header>
<nav class="navbar">
<div class="nav-container">
<div class="nav-logo">
<a href="{% url 'index' %}">👤 Employee Management</a>
</div>
<ul class="nav-menu">
<li><a href="{% url 'index' %}" class="nav-link">Dashboard</a></li>
<li><a href="{% url 'insert_works' %}" class="nav-link">Add Work Record</a></li>

```

```

<li><a href="{% url 'insert_lives' %}" class="nav-link">Add Location</a></li>
<li><a href="{% url 'search_company' %}" class="nav-link nav-highlight">Search by
Company</a></li>
<li><a href="{% url 'view_all_works' %}" class="nav-link">View All Works</a></li>
<li><a href="{% url 'view_all_lives' %}" class="nav-link">View All Lives</a></li>
</ul>
</div>
</nav>
</header>

<main>
<div class="container">
{% if messages %}
<div class="messages">
{% for message in messages %}
<div class="alert alert-{{ message.tags }}">{{ message }}</div>
{% endfor %}
</div>
{% endif %}

{% block content %}{% endblock %}
</div>
</main>

<footer>
<p>&copy; 2026 Employee Management System. All rights reserved.</p>
</footer>
</body>
</html>

```

Index.html

```

{% extends 'employees/base.html' %}

{% block title %}Dashboard - Employee Management{% endblock %}

{% block content %}
<div class="dashboard">
<h1>Employee Management System</h1>
<p class="subtitle">Manage employee work records and location information</p>

<div class="stats-grid">
<div class="stat-card">

```

```

<div class="stat-icon">📅</div>
<div class="stat-value">{{ works_count }}</div>
<div class="stat-label">Work Records</div>
</div>
<div class="stat-card">
<div class="stat-icon">🏠</div>
<div class="stat-value">{{ lives_count }}</div>
<div class="stat-label">Location Records</div>
</div>
<div class="stat-card">
<div class="stat-icon">🏢</div>
<div class="stat-value">{{ companies }}</div>
<div class="stat-label">Companies</div>
</div>
</div>

<div class="quick-actions">
<div class="action-card">
<h3>+ Add Work Record</h3>
<p>Insert employee work information (name, company, salary)</p>
<a href="{% url 'insert_works' %}" class="btn btn-primary">Go to Form →</a>
</div>

<div class="action-card">
<h3>🏠 Add Location</h3>
<p>Insert employee location information (street, city)</p>
<a href="{% url 'insert_lives' %}" class="btn btn-primary">Go to Form →</a>
</div>

<div class="action-card">
<h3>🔍 Search by Company</h3>
<p>Find employees by company and view their cities</p>
<a href="{% url 'search_company' %}" class="btn btn-primary">Go to Search →</a>
</div>
</div>
</div>
{% endblock %}

```

Search.html

```
{% extends 'employees/base.html' %}
```

```

{% block title %}Search by Company{% endblock %}

{% block content %}
<div class="search-page">
<div class="search-header">
<h1>Search Employees by Company</h1>
<p>Find employees working for a company and view their city locations</p>
</div>

<div class="search-container">
<form method="post" class="search-form">
{% csrf_token %}
<div class="search-group">
<label for="{{ form.company_name.id_for_label }}">Company Name:</label>
{{ form.company_name }}
{% if form.company_name.errors %}
<div class="form-error">{{ form.company_name.errors }}</div>
{% endif %}
</div>

<div class="search-actions">
<button type="submit" class="btn btn-primary">🔍 Search</button>
</div>
</form>
</div>

{% if results %}
<div class="results-section">
<h2>Search Results: {{ search_query }}</h2>
<p class="result-count">Found <strong>{{ results|length }}</strong>
employee(s)</p>

<div class="table-responsive">
<table class="data-table results-table">
<thead>
<tr>
<th>Person Name</th>
<th>Company Name</th>
<th>Salary</th>
<th>City</th>
<th>Street Address</th>
</tr>
</thead>

```

```

<tbody>
{% for result in results %}
<tr>
<td class="person-name">
<strong>{{ result.person_name }}</strong>
</td>
<td>{{ result.company_name }}</td>
<td class="salary">${{ result.salary|floatformat:2 }}</td>
<td class="city">
{% if result.city != 'Not available' %}
<span class="badge badge-info">{{ result.city }}</span>
{% else %}
<span class="text-muted">{{ result.city }}</span>
{% endif %}
</td>
<td class="street">{{ result.street }}</td>
</tr>
{% endfor %}
</tbody>
</table>
</div>
</div>
{% elif search_query %}
<div class="empty-state">
<p>No employees found for "{{ search_query }}"</p>
</div>
{% else %}
<div class="info-box">
<h3>How to Search:</h3>
<ol>
<li>Enter a company name in the search field</li>
<li>Click the "Search" button</li>
<li>View all employees from that company</li>
<li>See employee salary and city location information</li>
</ol>
</div>
{% endif %}
</div>
{% endblock %}

```


127.0.0.1:8000/admin/employees/lives/

127.0.0.1:8000/admin/employees/lives/

Django administration

WELCOME, ADMIN | [VIEW SITE](#) | [CHANGE PASSWORD](#) | [LOG OUT](#)

Home » Employees » Lives

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

Groups [Add](#)

Users [Add](#)

EMPLOYEES

Lives [Add](#)

Works [Add](#)

Select lives to change

ADD LIVES

Filter

Show counts

By city

All

Bangalore

Delhi

Mumbai

Noida

Pune

By created at

Any date

Today

Past 7 days

This month

This year

Action: Go 0 of 6 selected

☐	PERSON NAME	COMPANY	CITY	STREET	CREATED AT
☐	Yash Chauhan	Amazon	Noida	J-706	March 25, 2026, 9:53 a.m.
☐	Vikram Gupta	Infosys	Bangalore	Whitefield	March 25, 2026, 9:52 a.m.
☐	Neha Sharma	HCL Technologies	Pune	Kalyani Nagar	March 25, 2026, 9:52 a.m.
☐	Arjun Patel	Wipro	Bangalore	MG Road	March 25, 2026, 9:52 a.m.
☐	Priya Singh	TCS	Delhi	Dwarka	March 25, 2026, 9:52 a.m.
☐	Raj Kumar	Infosys	Mumbai	Bandra	March 25, 2026, 9:52 a.m.

6 Lives

127.0.0.1:8000

127.0.0.1:8000

Employee Management

Dashboard

Add Work Record

Add Location

Search by Company

View All Works

View All Lives

Employee Management System

Manage employee work records and location information

9

Work Records

+ Add Work Record

Insert employee work information (name, company, salary)

Go to Form

6

Location Records

+ Add Location

Insert employee location information (street, city)

Go to Form

7

Companies

Search by Company

Find employees by company and view their cities

Go to Search

© 2026 Employee Management System. All rights reserved.

Employee Management

DashboardAdd Work RecordAdd LocationSearch by CompanyView All WorksView All Lives

Search Employees by Company

Find employees working for a company and view their city locations

Company Name:

Amazon

Search

Search Results: Amazon

Found 1 employee(s)

Person Name	Company Name	Salary	City	Street Address
Yash Chauhan	Amazon	\$100000.00	Noida	J-706

© 2025 Employee Management System. All rights reserved.